EE2016 Microprocessors Theory and Lab, Aug - Nov 2025.

Tutorial 3 (Classes on 13, 15, 18, 19 & 20th of Aug, 2025) Solutions

Dr. R. Manivasakan, EE Dept, IIT Madras.

Portions: Wk5-Start ALU, shift register, flag register and their interconnection through address, control and data lines, ALU - components (logical & arithmetic operation engines), FPU, addressing modes (immediate, direct, indirect, register direct, register indirect), illustration of MOV in hardware, LDI ditto, assembly language CU, stack pointer, link register, EU & CU put together, AVR hardware architecture (Mazidi), RISC-style & CISC-style assembly programming.

1 Fill in the blanks

1. Instruction size is `fixed` (fixed / variable) in AVR.
2. The number of general purpose registers (GPRs) within the AVR processor is `32.`
3. The D7 bit of SREG in AVR denotes `I-bit denoting the global interrupt enable.`
4. In mask ROM, the contents are programmed by the manufacturer (manufacturer / user).
5. In programmable ROM, the contents are programmed by the `user` (manufacturer / user).
6. In EPROM, selectively a byte `cant` (can / cant) be erased and rewritten.
7. In Electronically Erasable PROM (EEPROM), selectively a byte `can` (can / cant) be erased and rewritten.
8. Flipflops are used to construct `SRAM` (SRAM / DRAM).
9. Capacitors are used to construct `DRAM` (SRAM / DRAM).
10. Volatile (S)RAM is made up of `FFs` (FFs / capacitors) and is made to be non-volatile by using `batteries`.
11. The advantage(s) of DRAM over SRAM is (are) `(a) cheaper (b) power consumed is very less`.
12. The advantage(s) of SRAM over DRAM is (are) `faster`.
13. The pair R27 R26 hold `16` bit address in the main memory and could be used to uniquely pull one location out of 2^{16}`memory` locations.
14. Stack pointer (SPH & SPL) is located in `I/O registers area` (GPR region or I/O registers area).
15. Enumerate the difference between IN and LDS instructions.
16. Instructions that affect all the six flag bits C, H, Z, S, V and N are `ADD, SUB` (ADD, AND, DEC, SUB, INC).
17. The difference between arithmetic shift left and logical shift left is (are) `none`.
18. Every assembly instruction consist of two parts: first part is `OP-CODE` and the second one being `operand field`

19. The EUCI doesnt (does / doesn't) change the sequence of program execution.

20. The PFCI does (does / doesn't) change the sequence of program execution.

21. The hardwired control unit consists of finite cyclic pulse train generator, instruction decoder and control circuit matrix.

2 Answer the following

1. The address bus of a computer has 16 address lines, $A_{15} - A_0$. The hexadecimal address assigned for a given device, is 7CA4.

   (a) Draw the address decoder (digital circuit). [Is it combinational circuit only or it is a combination of sequential & combinational logic?]

   (b) At a later point in time if the above address decoder is modified such that the device ignores lines A_8 and A_9, what are all the addresses to which this device will respond? [Mention the range of addresses].

   Solutions: (a) Consider a 16-bit AND gate with bits (given that 0x7CA4 is binary 0111 1100 1010 0100) $A_{15}, A_9, A_8, A_6, A_4, A_3, A_1, A_0$ are bubbled inputs and rest are directly connected. Output is connected to the CS (Chip Select) of the device. (b) The range (of second nibble from MSB) in Hex is 0x7CA4 till 0x7FA4 (meaning 0x7CA4, 0x7DA4, 0x7EA4, 0x7FA4, only these 4 words) for which the device responds.

2. Consider the problem of usage of stack and stack pointer in the form implemented in AVR ATmega8. Recall the picture on stack and stack pointer. Information:

   (a) The following is the list of data PUSHed onto stack during an interrupt (a) PC value (2 bytes) (b) SREG (1 byte) (c) GPRs (3 bytes) (d) stack pointer itself (2 bytes) (e) context data (function arguments only for subroutines, 2 bytes) (f) local variables (2 bytes). Thus, the stack frame for a subroutine call (SC) is 12 bytes in length, while for an interrupt, it is 10 bytes, which are PUSHed upon entry and POPed out upon from their 'exit'.

   (b) Stack frame is memory allocated in bytes, on the stack for a single subroutine call or interrupt call.

   (c) Stack Overflow: A critical error that occurs when a function call (or interrupt) attempts to PUSH data into the stack, but there is no more memory available.

   (d) Stack Capacity: The total available memory for the stack. Assume the stack in AVR 128 bytes

   The following sequence(s) of interrupts or subroutine combinations are presented to the AVR microcontroller. Legend: CS - Subroutine call, CI0 - AVR interrupt INT_0, CI1 - AVR INT_1, CI2 - AVR INT_2, R - return (from both interrupt and subroutine), RI - RETI in AVR

   (a) CS CS R CS R R

   (b) CI2 CS CI1 CS CS R R RI CI0 RI R CS R CS R RI

   (c) CI2 CS CS CS CI1 CS CS CS CI0 CS CS CS R R R

   Problem Statement: For each sequence above,

   (a) Write an algorithm to determine the maximum stack depth reached during the execution of each of the above three given sequences. You also need to detect if a stack overflow occurs at any point.

   (b) Provide

      i. maximum stack depth

      ii. final statement on whether a stack overflow has occurred.

   Hint: Use the following variables - Variable definitions:

- `current_depth`: an integer to track the deviation of stack pointer from the top location (position) [convention is that top most word has address 0x 0000 and address keeps incrementing, when you move downwards and reach last location 0x FFFF. The stack is defined in between this range].
- `max_depth`: an integer to keep track of maximum deviation of stack pointer from the top location
- `stack_capacity`: 128 bytes
- `frame_sizeS`: 12
- `frame_sizeI`: 10

Solutions:

Algorithm:

(a) Initialize `current_depth = 0` and `max_depth = 0.`

(b) Iterate through the input event sequence through string of characters by string.

(c) If the character is CI2 or CI1 or CI0 (Interrupt):

  i. Increment `current_depth` by `frame_sizeI`.

  ii. Update `max_depth` if `current_depth` is greater than `max_depth`.

  iii. Check for stack overflow: If `current_depth` is greater than `stack_capacity`, a stack overflow has occurred. Stop and report the error.

(d) If the character is CS (Subroutine Call):

  i. Increment `current_depth` by `frame_sizeS`.

  ii. Update `max_depth` if `current_depth` is greater than `max_depth`.

  iii. Check for stack overflow: If `current_depth` is greater than `stack_capacity`, a stack overflow has occurred. Stop and report the error.

(e) If the character is `R` (Return):

  i. Decrement `current_depth` by `frame_sizeS`.

(f) If the character is `RI` (Return):

  i. Decrement `current_depth` by `frame_sizeI`.

(g) Ensure `current_depth` does not become negative (this would indicate an invalid sequence).

(h) After the loop, report the `max_depth` and the final status (overflow or no overflow).

Part (b)

(a) maximum stack depth

  i. CS CS R CS R R - 24

  ii. CI2 CS CI1 CS CS R R RI CI0 RI R CS R CS R RI - 56

  iii. CI2 CS CS CS CI1 CS CS CS CI0 CS CS CS R R R - 138 (overflow)

(b) final statement on whether a stack overflow has occurred.

  i. No

  ii. No

  iii. Yes (138 at 12th call - a subroutine)

3. **Polling and Interrupt:** Consider the problem of interfacing a keyboard with an intel microprocessor operating at 250 MHz, given that a maximum of 1 key press per 20 msec (recall keyboard debouncing problem). The above interfacing could be done in two ways: (1) using polling KIN bit in KBD_STATUS and (2) using interrupt through KIRQ bit (in KBD_CONT) implementation. To see the performance difference between the polling and interrupt, assume there are only two key presses within an interval of

160 ms. Given that the CPU is running an user program of 2 million instructions and each instruction (CISC) is taking 20 clock cycles, what is the total time for completion of whole program in both cases (of polling & interrupt). Assume polling cycle (polling pulse repetition interval) is 0.125 msec and polling pulse width being 2 μsec. Ignore the time of reading from the KBD_DATA register into the CPU (4 nsec) and the interrupt latency of 8 nsec.

Solutions:

Total time for executing the program $= 2 \times 10^6 \times 20 \times 4 \times 10^{-9} = 160$ msec. Polling overhead $= \frac{160}{0.125} \times 2 = 2560$ musec. Hence total execution time in polling $= 162560$ musec. In the case, of interrupt, it is $= 160000 + (0.004 + 0.008) \times 2 = 160000.024$ musecs. This shows extra time wasted while polling the device. [Note that I have changed the problem to more realistic values].

4. **Priority Among Interrupts:** A processor has seven interrupt-request lines, INTR1 to INTR7. Line INTR7 has the highest priority and INTR1 the lowest priority. Design a priority encoder circuit (show truth table, Karnaugh's map & expression for output(s)) that generates a 3-bit code representing the request with the highest priority. See https://en.wikipedia.org/wiki/Priority_encoder.

Solutions:

With 7 (or 8) interrupts, the corresponding minimization of Karnaugh's map is getting too much complicated. Instead, we consider the four interrupts INT0 all the way up to INT3, with INT3 being the highest priority and INT0 being the least. Below are the computations and corresponding circuit.

I_7 - top priority interrupt

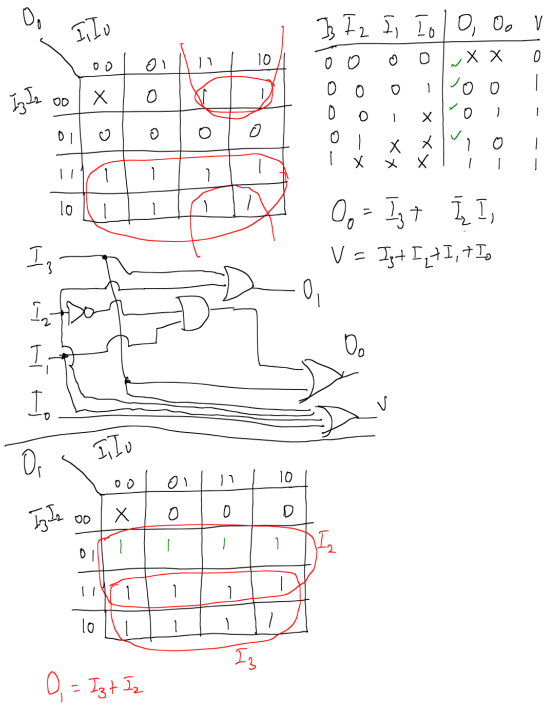
I_0 - least priority interrupt

V - whether O_i is valid or not (0- not valid, 1- valid)

$O_2O_1O_0$ → interrupt #. Eg. 010 → I_2, 111 → I_7

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	O_2	O_1	O_0	V
0	0	0	0	0	0	0	0	x	x	x	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	x	0	0	1	1
0	0	0	0	0	1	x	x	0	1	0	1
0	0	0	0	1	x	x	x	0	1	1	1
0	0	0	1	x	x	x	x	1	0	0	1
0	0	1	x	x	x	x	x	1	0	1	1
0	1	x	x	x	x	x	x	1	1	0	1
1	x	x	x	x	x	x	x	1	1	1	1

Minimization via Karnaugh map is quite involved. Instead consider 4 interrupts $I_3\ I_2\ I_1\ I_0$



I_3	I_2	I_1	I_0	O_1	O_0	V
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1